

Modern adatbázis rendszerek MSc – 11. Practice

MongoDB kezelése, API programozása - Java nyelv.

Folder: NEPTUNKOD_0429

Javasolt fejlesztőkörnyezet: Visual Studio Code

2. Collection létrehozása, adatok feltöltése

- a) Hozzon létre egy étterem nevű collection-t és adjon hozzá 3 éttermet. `_id` e5-től minden esetben legyen nagyobb

```
String connectionString = "mongodb://localhost:27017";

try (MongoClient mongoClient = MongoClient.create(connectionString)) {
    MongoDBDatabase database = mongoClient.getDatabase("VendeglatasDB");

    MongoCollection<Document> collection = database.getCollection("etterem");

    Document e1 = new Document("_id", "e6")
        .append("nev", "Gundel")
        .append("cim", new Document("varos", "Budapest")
            .append("utca", "Gundel Károly")
            .append("hazszam", "14"))
        .append("tipus", "Magyaros");

    Document e2 = new Document("_id", "e7")
        .append("nev", "Trattoria")
        .append("cim", new Document("varos", "Szeged")
            .append("utca", "Oskola utca")
            .append("hazszam", "10"))
        .append("tipus", "Olasz");

    Document e3 = new Document("_id", "e8")
        .append("nev", "Sakura")
        .append("cim", new Document("varos", "Debrecen")
            .append("utca", "Piac utca")
            .append("hazszam", "22"))
        .append("tipus", "Japán");

    collection.insertMany(Arrays.asList(e1, e2, e3));

} catch (Exception e) {
    System.err.println("Hiba történt: " + e.getMessage());
    e.printStackTrace();}
```

b) Olvassa be az ettermek.json és a szakacsok.json tartalmát.

```
MongoCollection<Document> etteremColl = database.getCollection("etterem");
String ettermekRaw = Files.readString(Paths.get("ettermek.json"));
List<Document> ettermekList =
Document.parse(ettermekRaw).getList("TempList", Document.class);

MongoCollection<Document> szakacsColl =
database.getCollection("szakacsok");
String szakacsokRaw = Files.readString(Paths.get("szakacsok.json"));
List<Document> szakacsokList =
Document.parse(szakacsokRaw).getList("TempList", Document.class);
```

a. Az éttermeket adja hozzá a már meglévő etterem collection-höz

```
List<Document> formattedEtteremList = ettermekList.stream().map(doc -> {
    String ekod = doc.getString("ekod");
    doc.put("_id", ekod);
    doc.remove("ekod");
    return doc;
}).collect(Collectors.toList());

etteremColl.insertMany(formattedEtteremList);
```

b. A szakácsoknak hozzon létre egy új collection-t, sz_kod legyen az _id

c. A szakácsok életkora legyen szám

```
List<Document> formattedSzakacsok = szakacsokList.stream().map(doc -> {
    String szKod = doc.getString("sz_kod");
    doc.put("_id", szKod);
    doc.remove("sz_kod");

    if (doc.containsKey("eletkor")) {
        String korStr = doc.getString("eletkor");
        doc.put("eletkor", Integer.parseInt(korStr));
    }
    return doc;
}).collect(Collectors.toList());

szakacsColl.insertMany(formattedSzakacsok);
```

3. Lekérdezések

a) Kérdezzük le az összes éttermet

```
System.out.println("--- a) Összes étterem lekérdezése ---");
List<Document> ettermek = etteremColl.find().into(new ArrayList<>());
for (Document doc : ettermek) {
    System.out.println(doc.toJson());
}
```

b) Kérdezzük le azt az éttermet, amelyek az azonosítója e3

```
System.out.println("\n--- b) Étterem lekérdezése (ekod: e3) ---");
Document e3 = etteremColl.find(eq("_id", "e3")).first();
if (e3 != null){
    System.out.println(e3.toJson());
}
```

c) Kérdezzük le azokat a szakácsokat, akiknek az életkora nagyobb mint 35

```
System.out.println("\n--- c) Szakácsok, akik idősebbek 35 évnél ---");
List<Document> szakacsok = szakacsColl.find(gt("eletkor", 35)).into(new
ArrayList<>());
for (Document doc : szakacsok) {
    System.out.println(doc.toJson());
}
```

d) Kérdezzük le azokat a szakácsokat és éttermüket, ahol a szakács életkora pontosan 40

```
System.out.println("\n--- d) Szakács (40 éves) és étterme (JOIN) ---");
List<Document> eredmeny = szakacsColl.aggregate(Arrays.asList(
    match(eq("eletkor", 40)),
    lookup("etterem", "e_sz", "ekod", "etterem_info")
)).into(new ArrayList<>());

for (Document doc : eredmeny) {
    System.out.println(doc.toJson());
}
```

e) Kérdezzük le a szakácsok átlagos életkorát

```
System.out.println("\n--- e) Szakácsok átlagos életkora ---");
Document atlag = szakacsColl.aggregate(Arrays.asList(
    group(null, avg("atlagEletkor", "$eletkor"))
)).first();
System.out.println("Átlagéletkor: " + atlag.get("atlagEletkor"));
```

4. Módosítás és törlés

- a) Módosítsuk az e2 azonosítóval rendelkező étterem nevét
- b) Töröljük ki az e4 azonosítóval rendelkező éttermet
- c) Töröljük ki azokat a szakácsokat, akiknek az életkora kisebb, mint 35

```
etteremColl.updateOne(  
    eq("_id", "e2"),  
    new Document("$set", new Document("nev", "Új Étterem Név"))  
);  
  
etteremColl.deleteOne(eq("_id", "e4"));  
  
szakacsColl.deleteMany(lt("kor", 35));
```